3.2 Discord与Slack接入

> **生成模型**：gpt-5.4 (openai/gpt-5.4)
> **Token 消耗**：输入 ~18k tokens，输出 ~4.5k tokens（本节）

如果说 Telegram 是“最容易迈出去的第一步”，那 Discord 和 Slack 更像“真正进入多人协作场景”。它们都很适合把 OpenClaw 放进团队讨论、值班频道、知识问答或者项目协作里，但接入思路和 Telegram 有几个明显区别。

最重要的一点是：**Discord 和 Slack 都不是单纯“拿个 token 就完事”的平台。** 你得先在平台后台创建应用，明确机器人身份，再把它邀请进服务器或工作区，最后还要处理权限、事件订阅、命令和线程这些细节。

这一节我们就做三件事：

1. 把 Discord Bot 从零建出来，接到 OpenClaw
2. 把 Slack App 的双 token 模式讲明白，并给出可跑配置
3. 顺带看一下 WhatsApp、Signal、iMessage 这些别的通道，大概是什么接法

3.2.1 先说 Discord：它为什么适合做 OpenClaw 通道

Discord 的优势不只是“年轻人常用”。从 AI 助手的角度看，它有几个很实在的优点：

- 服务器、频道、私信结构清楚
- Bot 能力成熟，权限粒度细
- Slash Commands 生态很适合做命令型交互
- 既能私聊，也能进群组频道，还能做线程化讨论

如果你是想做“团队里的 AI 助手”，Discord 基本是很自然的一站。

3.2.2 Discord 里几个你会反复见到的词

Application

在 Discord 里，Bot 不是凭空冒出来的。你得先在开发者后台创建一个应用。这个应用像个总壳，Bot 只是它下面的一个能力。

Bot Token

和 Telegram 类似，Discord Bot 也有 token。这个 token 就是 OpenClaw 登录 Discord 的关键凭证。

Guild

就是 Discord 的“服务器”。很多中文教程会直接说服务器，这样理解就够了。

Intent

可以理解成“你要告诉 Discord，Bot 打算关心哪些事件”。如果没开对应 Intent，某些消息即使真的发生了，Bot 也收不到。

尤其要注意 `Message Content Intent` 。如果你希望 Bot 读到普通文本消息内容，这项往往很关键。

3.2.3 第一步：创建 Discord Application 和 Bot

操作路径基本是这样：

1. 打开 Discord Developer Portal
2. 点击 `New Application`
3. 给应用起一个名字，比如 `OpenClaw Team Bot`
4. 创建完成后，进入左侧 `Bot`
5. 点击 `Add Bot`
6. 确认创建

创建好以后，你会在 Bot 页面看到用户名、头像、Token 管理等信息。这里最重要的是生成并复制 Bot Token。

注意：Discord 现在通常不会一直明文展示旧 token。你需要在后台重新生成或点击显示后立刻保存。操作风格跟很多云服务 API Key 很像：给你一次看清楚的机会，后面就别指望平台替你记着了。

截图说明建议

- 图 3-8：Discord Developer Portal 新建 Application 界面
- 图 3-9： `Bot` 页面点击 `Add Bot`
- 图 3-10：生成 token 后的页面，token 需打码

3.2.4 第二步：开启必要的 Intent

这是 Discord 接入里最容易漏的一步。

进入 Bot 设置页面后，通常要检查这些项：

- `MESSAGE CONTENT INTENT`
- `SERVER MEMBERS INTENT` （如果你涉及成员信息）
- `PRESENCE INTENT` （如果你确实需要在线状态）

对于大多数 OpenClaw 聊天场景，重点是第一项： `MESSAGE CONTENT INTENT` 。

不开它，Bot 往往就看不到你发的普通文本内容。结果就是：

- 机器人在线
- 权限也有
- 甚至能发消息
- 但就是像“听不懂人话”一样没反应

本质不是它不会回，而是它根本没读到正文。

3.2.5 第三步：邀请 Bot 进你的 Discord 服务器

Bot 在后台建好，还不等于它已经进到你的服务器里。你还得生成邀请链接，把它拉进去。

一般做法是：

1. 在 Developer Portal 里找到 OAuth2 / URL Generator
2. 勾选 `bot`
3. 如果你要用原生 Slash Commands，也勾选 `applications.commands`
4. 在权限里勾选最小必要集合

常见最小权限包括：

- View Channels
- Send Messages
- Read Message History
- Embed Links
- Add Reactions（如果你要做确认反馈）

复制生成的 URL，在浏览器打开，选择目标服务器，把它邀请进去。

截图说明建议

- 图 3-11：OAuth2 URL Generator 勾选 `bot` 和 `applications.commands`
- 图 3-12：浏览器中邀请 Bot 进服务器的确认界面

3.2.6 把 Discord token 写进 OpenClaw 配置

最小可跑的配置可以先这样写：

```
{
  channels: {
    discord: {
      enabled: true,
      token: "YOUR_DISCORD_BOT_TOKEN",
    },
  },
}
```

如果你希望命令体验更完整，可以再加一些和命令、群聊门控有关的项：

```jsonc
{
  channels: {
    discord: {
      enabled: true,
      token: "YOUR_DISCORD_BOT_TOKEN",

      commands: {
        native: true,
        prefix: "!",
      },

      mentionGating: true,

      allowlist: {
        guilds: ["123456789012345678"],
        channels: ["234567890123456789"],
        users: ["345678901234567890"],
      },

      maxLinesPerMessage: 30,
    },
  },
}
```

这里稍微解释一下。

`commands.native`

表示尽量利用 Discord 自己的原生命令体系，比如 Slash Commands。用户输入时会更像“平台原生功能”，而不是一个纯文本聊天机器人。

`prefix`

如果你还支持文本命令，这个前缀决定什么样的消息会被当成命令。比如 `!ask` 、 `!reset` 。

`maxLinesPerMessage`

这是偏实用的限制项。因为 Discord 单条消息虽然不短，但 AI 回复经常一大坨，做一点分段更稳，不然可读性会很差。

3.2.7 Discord 里怎么做第一次验证

建议按这个顺序测：

先测私信

直接给 Bot 发私信，问一句简单的话：

```
帮我写一个今晚的读书计划。
```

私信环境变量最少，排查最干净。如果私信都不通，先别进服务器里折腾。

再测服务器频道

在你刚邀请进去的频道里发：

```
@BotName 帮我总结一下这个频道最近的任务重点。
```

如果你开了 `mentionGating` ，那就必须明确提及它。没 `@` 而不回，未必是问题。

最后测命令

如果你开了原生命令或文本命令，再试：

```
!reset
```

或者在 Slash Commands 菜单里看命令是否注册成功。

3.2.8 Discord 常见问题，九成不是 OpenClaw 主逻辑坏了

情况一：Bot 在线，但死活不回

优先查：

- token 对不对
- Bot 是否真的被邀请进服务器
- `MESSAGE CONTENT INTENT` 是否开启

- 当前频道权限是否允许它看和发

情况二：私信能用，服务器频道不行

优先查：

- 频道权限单独被限制了
- 你开了 `mentionGating`，但没有 `@` 它
- 频道不在 allowlist 内

情况三：命令菜单没有出现

优先查：

- OAuth2 邀请时是否勾选 `applications.commands`
- 配置里是否启用了原生命令
- 平台同步有时有一点延迟，不一定是挂了

3.2.9 Slack 跟 Discord 最大的不同：它是 App 模式，不只是 Bot 模式

Slack 接入常常让新手更晕，因为它不是“注册个 Bot 就结束”。更准确地说，你是在创建一个 Slack App，然后这个 App 里包含 Bot 用户、事件订阅、Socket 连接、权限范围等一整套东西。

如果你只记一句话，那就是：**Slack 不是单 token 世界，它经常是双 token 甚至多凭证协作。**

3.2.10 先把 Slack 的两个核心 token 讲透

OpenClaw 接 Slack 时，常会提到两个 token：

Bot Token

通常长得像 `xoxb-...`，这是 Bot 身份调用 Slack API 的主要凭证。发消息、读频道、响应事件，很多动作都离不开它。

App Token

通常长得像 `xapp-...`，它更多用于 App 级连接，尤其是在 Socket Mode 里很常见。简单说，它让你的应用和 Slack 平台之间维持一条事件通信通道。

如果你看到“Bolt SDK + App Token + Bot Token”这组词，不要慌。它表达的其实是：

- 用 Slack 官方的 Bolt 框架接事件
- 用 App Token 建立应用级连接
- 用 Bot Token 执行具体聊天动作

3.2.11 第一步：创建 Slack App

基本步骤：

1. 打开 Slack API 后台
2. 点击 `Create New App`
3. 可以从零开始，也可以从 manifest 开始
4. 选择你的工作区
5. 创建后进入 App 配置页

然后你要做几件关键事：

- 打开 `Socket Mode`
- 生成 `App Token`
- 配置 `OAuth & Permissions`
- 安装 App 到目标工作区
- 生成 `Bot User OAuth Token`

截图说明建议

- 图 3-13：Slack API 后台创建新 App
- 图 3-14：Socket Mode 页面生成 `xapp-` token
- 图 3-15：OAuth & Permissions 页面生成 `xoxb-` token

3.2.12 Slack 需要哪些权限范围

Slack 里权限通常叫 `scopes` 。你可以把它理解成“这个 App 允许做哪些事”。

常见聊天场景至少会涉及：

- `app_mentions:read`
- `channels:history`
- `channels:read`
- `chat:write`

- `groups:history`
- `groups:read`
- `im:history`
- `im:read`
- `im:write`

你不一定每项都需要，但如果你要同时覆盖公开频道、私有频道、私聊，那 scopes 不会太少。

这里和 Discord 的一个差别是：Discord 更像“勾权限 + 勾 Intent”，Slack 更像“配 scopes + 订阅事件 + App 安装”。

3.2.13 把 Slack 写进 OpenClaw 配置

下面是一个比较典型的 Slack 通道配置：

```
{
  channels: {
    slack: {
      enabled: true,
      appToken: "xapp-1-A0000000000-0000000000-abcdefghijklmnop",
      botToken: "你的-slack-bot-token",

      mentionGating: true,

      allowlist: {
        channels: ["C0123456789"],
        users: ["U0123456789"],
      },

      threads: {
        enabled: true,
      },
    },
  },
}
```

这里的 `threads.enabled` 很值得注意。Slack 的实际使用里，线程是非常常见的协作单位。很多团队不希望 Bot 每次都把回复刷在主频道，而是希望它顺着某条消息在线程里接着聊。这样更整洁，也更符合工作区习惯。

3.2.14 Slack 里怎么验证是否接通

验证建议也按“先简单、后复杂”的顺序：

测试 1：私聊 Bot

给 Bot 发一句：

```
请帮我整理一下今天的待办。
```

测试 2：频道中 @ 提及

在频道里发：

```
@OpenClaw 帮我总结刚才讨论的上线风险。
```

测试 3：线程延续

如果它在某条消息下开了线程，再追问一句，看看上下文是否能顺下来。

Slack 场景里，线程连续性比“会不会回第一句”更重要。因为团队协作的真实对话，往往不是一轮结束，而是在线程里来回追问。

3.2.15 Discord 和 Slack，到底谁更适合你

如果你是个人项目、兴趣社区、开发者群体，Discord 往往更顺手。它天然适合：

- 公开频道
- 社区问答
- 命令式交互
- 开发协作

如果你是公司内部协作、日常办公、项目跟踪，Slack 通常更自然。它天然适合：

- 团队工作流
- 线程讨论
- 内部通知和知识问答
- 与其他办公系统串联

不是说一个比另一个“高级”，而是使用环境不一样。你先看用户在哪，不要倒过来看技术炫不炫。

3.2.16 其他几个常被提到的通道，先知道门路就够了

这一章重点不是把所有通道都讲到可部署，但你至少得知道 OpenClaw 的多通道世界长什么样。

WhatsApp

常见接法会提到 **Baileys**。它不是 WhatsApp 官方 Bot 平台那一路，而是围绕 WhatsApp Web 协议的一套实现。优点是灵活，接近真人账号使用体验；缺点是运维和稳定性会更微妙，也更依赖协议变化。

Signal

常见方案会依赖 `signal-cli` 一类工具链。它更像“把命令行或桥接程序接到消息系统里”，不是那种纯网页后台点几下就完事的通道。

iMessage

这一类通常和宿主系统绑定很深。你会看到像 BlueBubbles、macOS 原生桥接之类方案。它的难点不在 token，而在运行环境、设备权限和系统耦合。

所以从上手顺序来说，最推荐还是：

1. Telegram
2. Discord
3. Slack
4. 再看 WhatsApp / Signal / iMessage

这个顺序的原因不是功能强弱，而是成功率和排错成本。

本节小结

1. Discord 接入的关键不只是 Bot Token，还包括 Application 创建、服务器邀请和 Intent 开启。
2. `Message Content Intent` 对很多 Discord 聊天场景是关键项，不开它，机器人可能根本读不到正文。
3. Slack 更像 App 集成而不是单纯 Bot 集成，常见是 `App Token + Bot Token` 一起使用。
4. Slack 的线程机制非常适合协作型对话，配置时最好把线程场景一起考虑进去。
5. WhatsApp、Signal、iMessage 也都能接，但它们更依赖桥接层或宿主环境，建议等前面两个主流通道跑顺以后再扩展。